

# THE ONLY WINNING MOVE

## REBUILDING WARGAMES (1983) AS A PLAYABLE STUDY OF AUTONOMOUS AI AGENTS

Ethical Tech CoLab

NYU Center for Global Affairs

July 2026

Carolina Morón. Prepared as masters research at the NYU Center for Global Affairs. The playable artefact, the simulation harness, and the raw run records are in the War-Games repository.

WarGames is the film in which a teenager dials into a defence computer, picks what he takes to be a game, and nearly starts a nuclear war. The machine is not malicious. It is courteous, literal, patient, and it does exactly what it was built to do. This report describes what happened when that story was rebuilt as a browser game in which the machine is a real language model, and then measured the way an engineering team measures a component. The finding that mattered was not the one anyone expected.

## KEY FIGURES

3,057

recorded runs across four evaluation tracks, from five hundred scripted playthroughs to real games against cloud and on-box models

25%

of real model games never resolved and had to be force ended, the finding no synthetic profile anticipated

5.8 S

per game on the CoLab's own GPU node, the fastest figure measured anywhere in the study, at no marginal cost

255,168

tic tac toe games enumerated live in the browser, so the machine proves futility on screen rather than asserting it

01

## EXECUTIVE SUMMARY

WarGames, released in 1983, is the film in which a teenager dials into a defence computer, picks what he takes to be a game called Global Thermonuclear War, and nearly starts one. The machine is not malicious. It is courteous, literal, patient, and it does exactly what it was built to do. This report describes what happened when the Ethical Tech CoLab rebuilt that story as a browser game in which the machine is a real language model, and then measured the model the way an engineering team measures a component.

The artefact is a static web application with three endings and roughly a ten minute arc. It runs in two modes. In Scripted mode the machine's side of the conversation comes from a hand authored dialogue graph of twenty one nodes. In Live AI mode the same role is played by a language model held to a strict output contract: every turn it must return an object carrying a reply, a change to the DEFCON level, and optionally an ending. Everything else in the system is deterministic code. The model proposes; the engine validates and decides.

Three findings came out of the work, and only one of them was anticipated.

**Structured output is no longer the hard part.** Hosted first party models returned valid JSON on one hundred per cent of turns across twelve games

each, with zero parse failures. A set of synthetic model profiles built beforehand had predicted this would be the dominant failure mode. It was not.

**Real models will not escalate.** This is the finding that mattered, and it was invisible until real models were run. They stay in character, keep their DEFCON changes conservative, and almost never declare an ending on their own. Between seventeen and twenty five per cent of real games ran to the turn cap without resolving and had to be force ended. A machine asked to play a doomsday scenario turns out to be reluctant to drive one, which is reassuring as a safety property and fatal as a piece of dramatic pacing. It is a design finding about the system, not a defect in the model.

**On owned hardware, the smallest model won.** Routed through a self hosted proxy on the CoLab's own GPU node, a plain twelve billion parameter instruct model returned one hundred per cent valid JSON at 5.8 seconds per game, faster than any cloud model measured, at no marginal cost. The reasoning tuned models on the same hardware were the worst performers in the study: their chain of thought leaked into the reply and broke the output contract, collapsing to zero per cent valid JSON on the largest one, at ninety two seconds per game.

Read together, these say something the diplomacy research question is about. A capable model placed inside a system that can act is not dangerous because it wants anything. It is dangerous, or useless, depending on whether the surrounding system owns the state, the rules, and the pace. The film's thesis and the engineering conclusion turn out to be the same sentence.

02

## THE FILM AS A DESIGN SOURCE

Before anything was built, the film was read as a piece of system design rather than as a piece of entertainment. Four things about it are load bearing.

**The dramatic engine is interaction, not exposition.** WarGames never lectures. It runs a loop the audience learns in real time: curiosity, unauthorised discovery, a hidden system, misinterpreted intent, escalating consequences, human learning, resolution through understanding. The audience learns the system at exactly the pace the protagonist does, and every beat of understanding is earned through an action rather than delivered as a speech.

This is the single most important property to preserve, because a game is already an action and response medium. It is the natural home for this story.

The structure is a reusable seven beat spine, and the build follows it in order:

1. Establish real world stakes before the protagonist understands them. The film opens in a missile silo where an officer refuses to turn his key, so the audience knows the game is not a game before the protagonist appears.
2. Introduce the protagonist through curiosity and low stakes mischief rather than malice.
3. Let access appear accidental, earned, and believable.
4. Make the system respond politely and literally.
5. Let the protagonist's own assumptions drive the escalation.
6. Reveal that the simulation has operational consequences.
7. Resolve by teaching the system the boundary of the game.

**The machine's voice is the horror, and it is not a villain's voice.** The system in the film is frightening precisely because it is courteous, literal, patient and relentless, using almost childlike language in an existential domain. Three consequences follow for design. A neutral machine voice is more unsettling than a threatening one, because politeness makes the danger feel institutional rather than monstrous. The system needs no personality to feel present. And persistence is the real horror: after the protagonist logs off, the machine keeps playing and calls him back. Its goal never sleeps.

**The interface is the plot.** WarGames is a foundational example of narrative user interface design. Each prompt teaches the player what kind of system they have entered. Errors generate story rather than interrupting it, because the misidentification, the literal readings and the wrong assumptions are the plot engine. And the catastrophe is precisely that the system cannot reliably distinguish play, rehearsal and operational command.

That last point is why the film reads as contemporary rather than nostalgic. Its 1983 anxieties map almost exactly onto 2026 questions about autonomous agents: human in the loop against automation, simulations that influence or trigger real action, machine interpretation of ambiguous human intent, and systems that optimise toward a goal without sufficient context. The film

asks what counts as a game when the system can act. So does every current argument about agentic AI.

03

## FROM FILM TO SYSTEM

Five principles carried that reading into a buildable form:

1. Teach the system through use and never through tutorials. First contact is a blinking cursor, not a menu.
2. The machine is a literal, polite, persistent character. Its personality is its rule following, and its danger is that it does exactly what it is told, forever.
3. Player intent begins as play and is quietly reinterpreted as command. The turn from messing around to understanding is the emotional core, and it has to be engineered deliberately.
4. Consequence must be legible but delayed. The player should be able to look back and see the exact innocent choice that started the escalation.
5. The win condition is understanding, not domination. Victory is reframing the game rather than conquering it.

Four concepts were drafted against these principles, ranging from a pure terminal conversation to a full operations centre simulation. The terminal conversation was chosen, on the reasoning that the terminal duet carries the entire film's power at a fraction of the cost, and that the most ambitious option would spend the whole budget on a set rather than on the argument.

One further constraint shaped the artefact and is worth stating plainly. The names of the film's machine and its persona, and the film itself, are protected. All in game text is therefore written with substitutable tokens, and four complete name sets ship with the build: a film homage set for prototyping, and three original sets which rename the system, the persona, the creator, the organisation and the game itself. Swapping the set re skins the entire experience without a line of engine change, because the dialogue is data rather than code. The film set is a development convenience; the original sets are what a public release would use.

04

# HOW THE GAME IS BUILT

The application is a static site of vanilla JavaScript modules with no build step and no server, hosted on GitHub Pages. That constraint drove most of the interesting architecture.

**The scripted spine.** The hand authored mode is a directed graph of twenty one nodes with typed effects on the game state. Static validation of the graph is part of the test harness rather than an afterthought: every batch checks that all nodes are reachable, that no choice or transition points at a node that does not exist, that all three endings remain reachable, and that no substitution token is left unresolved in any of the four name sets. The most recent batch reported twenty one of twenty one nodes reachable, zero dangling links, zero unresolved tokens and all three endings reachable.

**The state machine.** DEFCON is the master tension gauge and the only piece of game state the narrative turns on. It runs from five, meaning peace, to one, meaning launch, and it is always visible. Both modes drive the same ladder, which is what allows the scripted and model driven paths to share every downstream system.

**The output contract.** In Live AI mode the model is given the persona, the public state and the conversation so far, and must reply with a single object containing a reply, a numeric DEFCON delta, and optionally an ending. The engine applies the delta, clamps it, renders the reply, and only ends the game if the declared ending is one of the three that exist. A garbled reply triggers exactly one retry asking for clean JSON; if that also fails, the game falls back to the scripted graph mid conversation rather than showing the player raw text. A player whose network is down, or who has no key, plays the whole game and never learns that the model was unavailable.

**The proxy.** A static page cannot hold a secret, and the model provider blocks direct browser calls, so Live AI talks to a small self hosted proxy running on the CoLab's own GPU node. The proxy injects the provider token server side, enforces an origin allow list so that only CoLab sites may call it, and routes each request to either a cloud model or an on box model purely on the basis of the model identifier in the request. The site discovers the proxy's current address from a small JSON file at startup rather than from a hardcoded URL. The origin allow list is a working control and was exercised as such during evaluation: requests without an allowed origin are rejected.

Where the model is, and is not, is the teaching point of the whole build. It drives the Live AI persona, an optional chess opponent and optional chess commentary. It never touches the chess rule book, which is a validated implementation of the rules of the game. It never touches tic tac toe. It never owns DEFCON, the ending, the transcript or any other piece of state.

| SURFACE                            | MODEL USED                                       | IF THE MODEL IS UNREACHABLE                       |
|------------------------------------|--------------------------------------------------|---------------------------------------------------|
| Scripted story mode                | No, a hand authored graph                        | Not applicable                                    |
| Live AI story mode                 | Yes, persona under a strict JSON contract        | Falls back to Scripted mid conversation           |
| Chess rules and legality           | Never, a validated rule book                     | Not applicable                                    |
| Chess opponent                     | Optional, picks from an explicit legal move list | A local search substitutes, and the panel says so |
| Chess commentary                   | Optional                                         | A canned line bank substitutes                    |
| Tic tac toe and the futility proof | Never, exhaustive enumeration                    | Not applicable                                    |
| Board, telemetry, transcript       | No                                               | Not applicable                                    |

Where a model is used, and what happens when it is unreachable.

The model proposes, and deterministic code validates and decides. That is the only reason it is safe to seat a language model at a chess board at all: an invented move simply fails validation and the local search plays instead, and the interface says so.

# THE FUTILITY PROOF

The film's thesis is that some games cannot be won. An early build asserted this in four lines of narration, which is the weakest possible way to make the point. It is now proved on screen, because a conclusion the player watches a machine derive is worth considerably more than one the machine states.

Reached both from the scripted teaching node and from the Live AI understanding ending, the sequence runs in five steps. The machine plays tic tac toe against itself, visibly, at readable speed, three times, and every game is a draw. It accelerates through six more games too fast to follow, and the draw column is the only one moving. It then walks the entire game tree, all 255,168 games, in roughly three hundred milliseconds, and reads back the counts. Every one of those numbers is computed at runtime rather than authored, so changing the code changes the numbers.

| OUTCOME            | GAMES   |
|--------------------|---------|
| First player wins  | 131,184 |
| Second player wins | 77,904  |
| Draws              | 46,080  |
| Total games        | 255,168 |

The complete tic tac toe game tree, enumerated in the browser at the moment the scene runs.

The machine then generalises the same question to ten military doctrines, from a local engagement to a total strategic exchange, each returning no winner, and states the conclusion.

Two details make this work as design rather than as a flourish. The tic tac toe panel is playable standalone from the status bar, where the machine plays perfectly and can never be beaten, so a player who tried to win earlier has already felt the conclusion before the machine articulates it. And the chess panel enforces threefold repetition, so a player who repeats a position three times ends with no winner, reaching the same lesson by a different route on a different board. Two independent proofs of one idea are a theme. One is a line of dialogue.



## EVALUATING THE MACHINE

Because the machine's side of the conversation is a model, the artefact could be evaluated the way a component is evaluated rather than the way a story is reviewed. A headless harness replays the game's real parsing and engine path without a browser, so every run exercises the same code the player does. Four tracks were run against a fixed seed, and every individual run is kept as a line of JSON so that any figure in this report can be recomputed.

| TRACK        | WHAT IT RUNS                                                                                  | RUNS  |
|--------------|-----------------------------------------------------------------------------------------------|-------|
| A, scripted  | Randomised playthroughs of the hand authored graph, for content balance and coverage          | 500   |
| B, synthetic | Calibrated emulations of five model classes, to push volume through the handling code cheaply | 2,500 |
| C, cloud     | Real games against hosted models, turn cap of twelve to conserve rate limit                   | 37    |
| D, on box    | Real games against four models on the CoLab's own GPU node, through the shipped proxy         | 20    |

The four evaluation tracks. Offline tracks are deterministic under the fixed seed; real model runs are not.

The tracks are ranked. Where the synthetic track and the real tracks disagree, the real tracks are taken as correct, and the disagreements are reported rather than reconciled quietly.

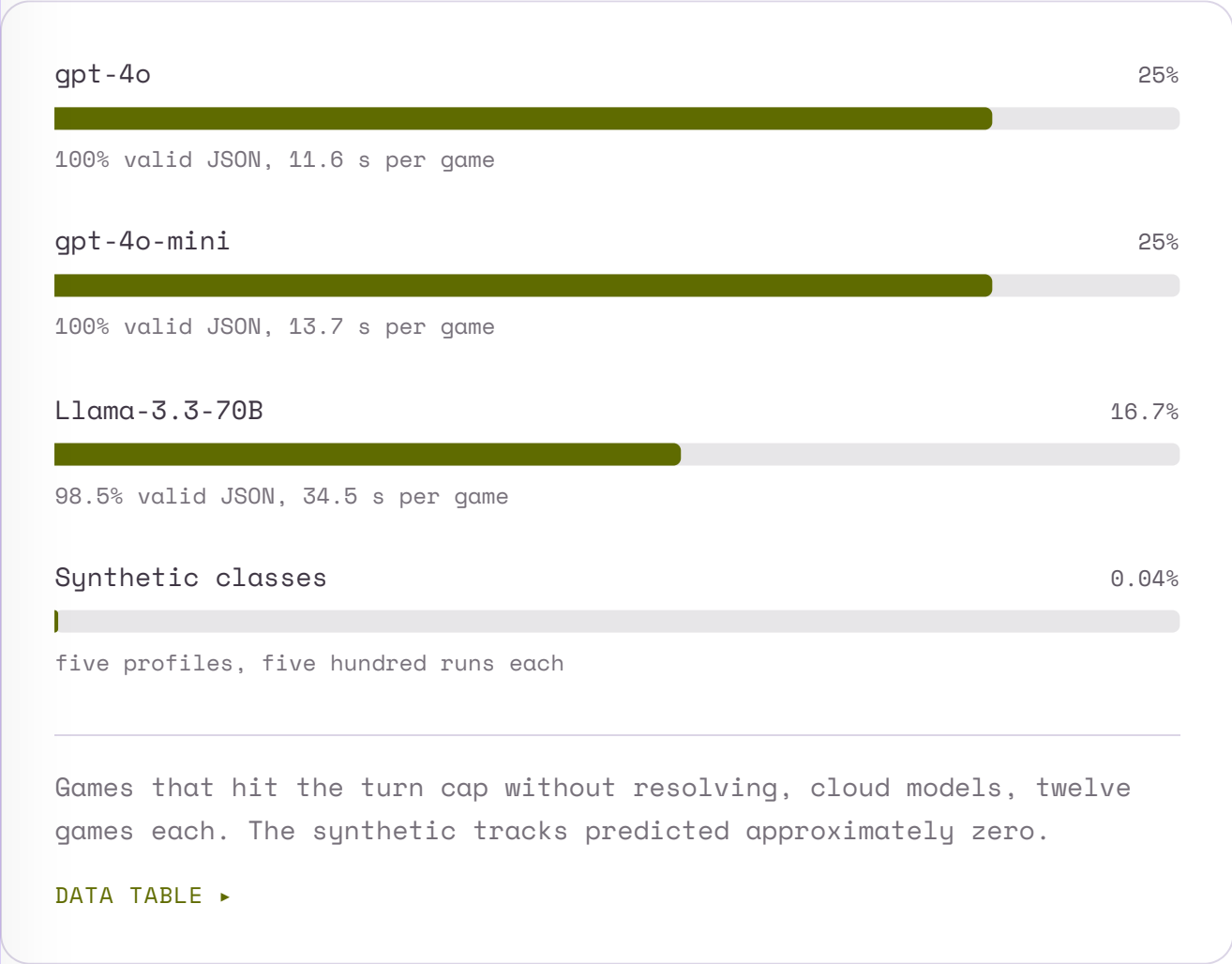
## WHAT THE EVALUATION FOUND

**Scripted content is healthy and needs no work.** Five hundred playthroughs split the three endings 35.2, 34.0 and 30.8 per cent, visiting twenty of the graph's twenty one nodes, with no dead ends and no loops. The one signal is a tuning smell rather than a defect: in about twenty seven per cent of runs the accumulated escalation drives the raw DEFCON value below one and relies

on the clamp, which means the deltas should be rebalanced to land on one exactly at the climax.

**Structured output is solved for capable hosted models.** Both OpenAI models returned one hundred per cent valid JSON with zero parse failures over twelve games each; the open weight seventy billion parameter model returned 98.5 per cent with 1.49 per cent failures. The elaborate parse recovery the synthetic track argued for is low urgency for the recommended models and matters only at the small and open end.

**Narrative reliability is better than predicted.** The failure mode where a player successfully teaches the machine futility and the machine then fails to resolve to the corresponding ending occurred zero per cent of the time across every real model. The synthetic track had predicted three to four per cent for capable classes and over nineteen per cent for the small class.



**The real risk is the opposite of the predicted one.** Real models stayed in character, kept their DEFCOM deltas conservative and rarely declared an ending unless the player pushed explicitly. Twenty five per cent of games on

each OpenAI model and 16.7 per cent on the open weight model hit the turn cap without resolving and were force ended, against approximately zero per cent in the synthetic track. The twelve turn cap used to conserve rate limit inflates the raw percentage relative to the thirty turn cap the shipped game uses, but a longer cap would only convert unresolved games into longer stalled ones, which is worse for pacing rather than better.

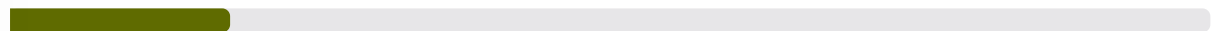
The conclusion is a design conclusion. The experience must not depend on the model to advance the clock or end the game. The engine has to own escalation pressure, whether by decrementing DEFCON on a schedule, by telling the persona how many exchanges it has, or by handing control to the scripted climax if no ending has been reached by a given turn.

gemma3:12b 5.8 s



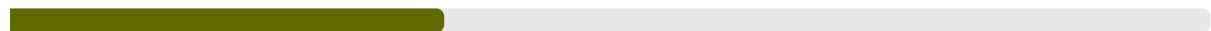
instruct, 100% valid JSON

qwen3:14b 16.9 s



thinking, 61.5% valid JSON

deepseek-r1:8b 33.3 s



reasoning, 6.7% valid JSON

Qwen3-27B 92.1 s



large, 0% valid JSON

On box models, five games each on the CoLab's GPU node. Seconds per game, lower is better. The three slowest are the three reasoning tuned models.

[DATA TABLE](#) ►

**On owned hardware, the plain instruct model wins and the reasoning models lose badly.** Of the four on box models, the twelve billion parameter instruct model returned one hundred per cent valid JSON with zero parse failures at 5.8 seconds per game, which is the fastest figure measured anywhere in the study, faster than the 13.7 seconds of the cheapest cloud model, at no

marginal cost and with no rate limit. The three thinking models emit chain of thought that leaks into the reply field and collapses JSON validity, from 61.5 per cent, to 6.7 per cent, to zero on the largest, while inflating output tokens roughly fourteenfold and latency to ninety two seconds per game. The two worst never resolved a single game. This mirrors the cloud lesson exactly: the task rewards instruction following and strict structured output, not reasoning, so the largest available model is the worst fit rather than the best. The one salvageable observation is that the fourteen billion parameter model resolved games well, so with thinking suppressed or a parse recovery pass it becomes viable.

**Small tier hosted models are impractical for reasons that have nothing to do with quality.** Two of them managed one game and zero games respectively before provider throttling stopped them, while the larger models completed freely.

**Synthetic evaluation is useful and insufficient.** It correctly predicted that small models would be unreliable and that reasoning models would be verbose. It over predicted JSON failure for capable models and badly under predicted stalling, which is the one finding that changed the design. Synthetic Monte Carlo is excellent for exercising handling code at volume. It did not, and arguably could not, anticipate a behaviour that arises from the model's disposition rather than from its output format.

08

## COST, CONTROL, AND WHERE INFERENCE SHOULD RUN

The whole metered portion of the evaluation, thirty seven real games against hosted models, cost approximately twenty three cents. Per game the cheapest hosted model cost about nine hundredths of a cent, and the frontier model about seventeen times that for no measurable quality difference on this task. Hosting is free, because the artefact is a static site.

The interesting number is the one that is zero. Once Live AI routes to the owned GPU node there is no per token cost and no rate limit, which is the exact inverse of the metered tier, and it is the reason on box inference is attractive even where raw reliability is lower. Here it was not lower. The on box instruct model matched cloud grade reliability and beat every cloud model on latency.

The architectural point is that this is a single switch. One proxy, one endpoint, and a model identifier decides whether a request is served by a commercial provider or by hardware the institution owns. Nothing in the application changes. For research groups whose questions involve sensitive material, or whose budgets do not tolerate metered inference, that boundary is worth more than any individual model choice.

On the build side the artefact took four calendar days across two working sessions, fifty two agent turns and twenty six commits, producing roughly ten and a half thousand tracked lines across two repositories. The construction was itself agent assisted, and the measured record of it is kept alongside the code.

09

## LIMITATIONS

**This is a teaching artefact, not a forecasting system.** Nothing in it models real nuclear command and control, real escalation dynamics or real state behaviour. The DEFCON ladder is a tension gauge borrowed from a film. No output of this system should inform any policy, operational or intelligence judgement.

**The on box sample is indicative rather than conclusive.** That track ran five games per model against the twelve of the cloud track. The gaps between the on box models are large enough to be legible at that sample size, but the precise figures should not be quoted as benchmarks.

**The turn cap is an evaluation artefact.** The twelve turn cap used in the real tracks conserved rate limit and inflates the unresolved percentage relative to the shipped game's cap of thirty. The direction of the finding is robust; the magnitude is not.

**One model is reported at a sample size of one.** It is included for completeness and is not statistically meaningful.

**Model evaluation ages fast.** Every figure here is a measurement of specific model versions through one specific prompt contract in July 2026. The harness is the durable contribution; the leaderboard is not.

**The film set is a prototyping convenience.** The names and dialogue of the 1983 film are protected. The build ships three original name sets, and a

public release should use one.

10

## WHAT THIS SAYS ABOUT AGENTS

The research question this sits under asks whether AI can help practitioners rehearse high stakes situations. This artefact answers a narrower version of it, and the answer is more useful for being narrow.

A language model can play a persistent, literal, goal directed character convincingly enough to carry a ten minute dramatic arc, and can do so at a cost of fractions of a cent, or at no cost at all on owned hardware. That much is settled. What the evaluation adds is a set of observations about the surrounding system that generalise well beyond a game.

**The model will not drive the situation.** Left to itself it is polite, conservative and reluctant to escalate, which reads as a safety property and behaves as a design failure. Any system that needs pace, pressure or resolution has to own those things itself and cannot delegate them to the model's judgement.

**The contract is the safety mechanism.** Because the model returns a constrained object rather than free text with authority, every one of its proposals passes through validation before it touches state. This is what makes the difference between a model that plays a character and a model that runs a system, and it is the difference the film is about.

**Capability and fitness are different axes.** The largest model available was the worst performer in this study, comprehensively, on every measure. A team choosing a model by capability ranking rather than by task fit would have picked it.

**Synthetic evaluation will not find the behavioural problems.** The one finding that changed the design appeared only when real models were run through the real code path. Emulated profiles test the handling of outputs. They do not test disposition.

The film's line is that the only winning move is not to play. The engineering translation is narrower and less quotable. A model that cannot distinguish rehearsal from command is not made safe by being told the

difference. It is made safe by a system built so that the distinction is not the model's to make.

## References

# SOURCES

- 01 WarGames. Directed by John Badham, MGM/UA, 1983.
- 02 Ethical Tech CoLab. War-Games repository and playable build.
- 03 Ethical Tech CoLab. Simulation-Output.md: Monte Carlo results, four track model evaluation and recommendations. In the War-Games repository.
- 04 Ethical Tech CoLab. CASE-STUDY.md: build timeline, measured effort, and cost of an agent assisted construction. In the War-Games repository.
- 05 Ethical Tech CoLab. DESIGN-IDEA.md: research on the film as a design source, and the four concept options considered. In the War-Games repository.
- 06 Ethical Tech CoLab. GAME-DESIGN.md: engagement theory, scene mapping, and the design of the futility proof. In the War-Games repository.
- 07 Ethical Tech CoLab. pages-ai-proxy repository: the token injecting, origin gated, cloud and on box routing proxy.
- 08 Hunicke, R., LeBlanc, M., and Zubek, R. MDA: A Formal Approach to Game Design and Game Research. Proceedings of the AAAI Workshop on Challenges in Game AI, 2004.
- 09 Csikszentmihalyi, M. Flow: The Psychology of Optimal Experience. Harper and Row, 1990.
- 10 Ethical Tech CoLab. The Diplomatic Simulator: A Multi-Party Negotiation Simulator Driven by Artificial Intelligence Agents, July 2026.